

Лекція 11. Огляд мінімально укомплектованої CI-системи

Що таке мінімально укомплектований CI?

Це базова інфраструктура для неперервної інтеграції (Continuous Integration), що включає репозиторій, сервер збірки та системи перевірки якості. Дозволяє автоматизувати збірку, тестування та аналіз коду.

Мета лекції:

- Розглянути базові компоненти мінімального CI.
- Пояснити принципи їх взаємодії.



Вимоги до CI-системи

CI-система має на меті забезпечення автоматизацію ключових процесів розробки та тестування програмного забезпечення і має на меті швидке виявлення помилок і забезпечення якості коду після регулярного інтегрування змін.

Така система може складатись з наступних базових компонентів:

- *системи контролю версій*: дозволяють відстежувати зміни в кодовій базі, керувати різними версіями програмного забезпечення та ефективно співпрацювати в команді;
- *сховища коду (GitHub, GitLab або Bitbucket)*: надають веб-інтерфейс для роботи з репозиторіями та додаткові інструменти для спільної розробки. Ці платформи забезпечують функціонал для code review, керування проектами, відстеження помилок та інтеграції з іншими інструментами розробки;
- *CI-інструмент*: дозволяє автоматизувати робочі процеси, включаючи збірку проекту, запуск тестів та розгортання;
- *тестове середовище*: використовуються рішення, які можуть бути реальними (фізичні сервери або локальні машини) чи віртуалізованими (контейнери Docker);

- *інструмент для аналізу коду*: перевіряє код на відповідність стандартам, виявляючи потенційні вразливості, code smells та дублювання коду;
- *система оповіщень (Slack, електронна пошта або звіти в CI-платформі)*: інформує команду про статус збірки, тести та результати аналізу коду.

CI-сервери, такі як Jenkins, CircleCI чи GitLab CI/CD виконують автоматизовані завдання, такі як збірка додатків, запуск тестів і перевірка якості коду. GitHub Actions, як вбудований інструмент CI/CD в GitHub, дозволяє автоматизувати робочі процеси безпосередньо в репозиторії, включаючи збірку, тестування та розгортання додатків.

Інструменти для аналізу коду (наприклад SonarQube), інтегрується в CI-процеси для перевірки коду на наявність технічного боргу, вразливостей і відповідність стандартам написання. Вони також відстежують покриття коду тестами та допомагають контролювати технічний борг.

Для централізованого зберігання та управління збірками проекту, бібліотеками і залежностями можна використати артефакт-репозиторії, такі як Nexus або JFrog Artifactory

Автоматичні нотифікації є важливою складовою CI/CD-процесів, оскільки вони дозволяють тримати команду в курсі подій. Автоматичні нотифікації забезпечують оперативність, дозволяючи членам команди миттєво отримувати інформацію про стан пайплайнів і швидко реагувати на помилки. Вони сприяють прозорості, тримаючи всю команду в курсі прогресу CI/CD-процесу, включаючи інформацію про успішні та провальні білди.

Інтеграція зі звичними інструментами дозволяє надсилати сповіщення через Email, Slack, Telegram, GitHub або інші системи, які команда використовує у своїй щоденній роботі. Нотифікації можна налаштувати для відображення лише певних подій, наприклад, лише для невдалих білдів або результатів проходження Quality Gate у SonarQube.

Сповіщення можуть містити детальні звіти, такі як лінки на результати білду, логи, статус Quality Gate чи метрики тестів, що дозволяє отримувати всю необхідну інформацію в одному місці.

Для зручності сприйняття нотифікації в Slack або Telegram можуть використовувати кольорове кодування, щоб позначати статус завдань: зелений для успіху та червоний для помилок.

Таким чином, CI-система є критичним інструментом сучасної розробки програмного забезпечення, що забезпечує автоматизацію, контроль якості та ефективну співпрацю команди. Вона дозволяє швидко виявляти та виправляти помилки, підтримувати високу якість коду та прискорювати процес розробки.

Правильно налаштована CI-система значно знижує ризики при розробці, покращує якість продукту та підвищує ефективність команди через автоматизацію рутинних завдань та забезпечення швидкого зворотного зв'язку.

11.2. Використання Jenkins для побудови CI-системи

Розглянемо використання інструментів GitHub, Jenkins та SonarQube для побудови мінімально укомплектованої CI-системи.

З'єднання GitHub репозиторію та Jenkins

Основні способи:

1. Webhooks:

- GitHub надсилає повідомлення про події (push, pull request) на Jenkins.

2. Полінг (Polling):

- Jenkins періодично перевіряє GitHub на наявність змін.



Webhook — механізм активного сповіщення про події в реальному часі.

У GitHub можна налаштувати webhook для виклику Jenkins при зміні репозиторію.

Переваги:

- Немає необхідності в постійному опитуванні серверів.
- Миттєва реакція на коміт.



Cron — спеціальна синтаксична форма для планування завдань у часі.

У Jenkins для налаштування регулярного запуску можна використовувати вирази:

- `H/15 * * * *` — кожні 15 хвилин.
- `@daily` — раз на день.

Використовується для резервного контролю, якщо webhook не спрацював.

Jenkins – це потужний інструмент для автоматизації процесів розробки, який часто використовується в тісній інтеграції з GitHub. Для підключення GitHub репозиторію до Jenkins існують кілька способів, кожен із яких має свої переваги та підходить для певних сценаріїв.

Для з'єднання з репозиторієм можна використати наступні способи:

- плагін Git Plugin
- Webhooks
- Polling SCM

Git Plugin

Jenkins має вбудовану підтримку Git через плагін Git Plugin, який дозволяє з'єднувати репозиторій із сервером Jenkins.

Для встановлення з'єднання можна виконати наступні кроки:

- 1) Встановити Git Plugin через «Manage Jenkins» → «Manage Plugins».
- 2) Додати репозиторій до Jenkins job: для цього у полі «Source Code Management» слід обрати Git та вказати URL репозиторію (HTTPS або SSH);
- 3) Налаштувати аутентифікацію:
 - **HTTPS:** використовуйте особистий токен доступу (Personal Access Token) для GitHub, який можна згенерувати у налаштуваннях GitHub;
 - **SSH:** використовуйте SSH-ключ, який додається до Jenkins у розділі «Credentials».

Приклад використання SSH:

```
git@github.com:username/repository.git
```

Webhooks

Webhooks – це механізм зворотного виклику, який дозволяє миттєво реагувати на події, що відбуваються в репозиторії (наприклад, push, створення pull request, merge). Ця технологія базується на передачі HTTP-запитів від GitHub

(чи іншого сервісу) до сервера Jenkins, завдяки чому інтеграція стає більш ефективною та динамічною. В даному випадку Webhooks дає змогу автоматично запускати Jenkins job при змінах у репозиторії.

Для налаштування потрібно проробити наступні кроки:

1) У налаштуваннях GitHub репозиторію потрібно перейти у розділ «Settings» → «Webhooks» та додайте новий webhook:

- URL: `http://<Jenkins_URL>/github-webhook/`
- Content type: `application/json`

2) Слід переконатись, що Jenkins має плагін "GitHub Integration Plugin" та активувати опцію «GitHub hook trigger for GITScm polling» у налаштуваннях Jenkins job.

3) Далі слід налаштувати webhook, вказавши події, на які буде реагувати webhook (наприклад, push, pull request, tag). Коли зазначена подія відбувається, GitHub надсилає HTTP POST-запит на вказаний URL. Jenkins приймає запит і запускає відповідний pipeline або job.

Нижче наведено приклад у Jenkins Pipeline:

```
groovy

pipeline {
    agent any
    triggers {
        githubPush() // Використання тригера GitHub Push через webhook
    }
    stages {
        stage('Build') {
            steps {
                echo 'Building project...'
            }
        }
    }
}
```

Перевагою цього способу є висока швидкість виконання, адже завдання запускаються майже миттєво після події у репозиторії. Це також допомагає знизити навантаження на сервер Jenkins, оскільки відсутня необхідність у постійних перевірках репозиторію.

Серед недоліків можна відзначити вимогу забезпечення публічного доступу до Jenkins, що потребує налаштування NAT, проксі-сервера або використання VPN для безпечного з'єднання.

Polling SCM

Polling SCM – це метод, за якого Jenkins періодично перевіряє стан репозиторію на наявність змін. Це підходить для випадків, коли налаштування Webhooks неможливе (наприклад, через обмеження мережі).

Для налаштування можна скористатись наступним алгоритмом:

- 1) У налаштуваннях Jenkins job увімкнути опцію "Poll SCM".
- 2) У полі розкладу (Schedule) використати cron-подібний синтаксис, наприклад:

```
H/5 * * * *
```

Цей запис означає перевірку репозиторію кожні 5 хвилин.

Перевагою цього підходу є його простота налаштування, особливо для середовищ із закритими мережами, де публічний доступ обмежений.

До недоліків належать додаткові витрати ресурсів на періодичні перевірки репозиторію, навіть якщо змін немає, а також можливі затримки між моментом внесення змін (push) і запуском збірок.

Порівняння способів інтеграції між GitHub та Jenkins

Спосіб	Швидкість реакції	Налаштування	Ресурси Jenkins	Рекомендоване використання
Git Plugin	Середня	Легке	Помірні	Базова інтеграція
Webhooks	Висока	Складніше	Мінімальні	Часті зміни у коді
Polling SCM	Низька	Легке	Високі	Закриті мережі

Для більшості сучасних сценаріїв інтеграції Webhooks є найкращим вибором, оскільки вони забезпечують миттєву реакцію на зміни в репозиторії без перевантаження сервера. Якщо у вас є специфічні вимоги, такі як обмежений доступ до інтернету, можна використовувати Polling SCM.

Використання cron-стилю в Jenkins Jobs

Jenkins підтримує різні підходи для автоматизації запуску завдань залежно від потреб проєкту. Окрім Webhooks, налаштувати реакцію системи на зміни в кодовій базі або планувати регулярне виконання завдань можна використовуючи cron-стиль

Cron-стиль використовується в Jenkins для завдань, які повинні виконуватися на регулярній основі або в умовах, коли webhooks недоступні. Планувальник Jenkins базується на синтаксисі cron – текстових виразах, які визначають періодичність виконання.

Синтаксис cron у Jenkins:

```
sql  
  
MINUTE HOUR DOM MONTH DOW
```

- MINUTE: хвилина (0-59);
- HOUR: година (0-23);
- DOM (Day of Month): день місяця (1-31);
- MONTH: Місяць (1-12);
- DOW (Day of Week): день тижня (0-7), де 0 і 7 – це неділя;

Приклади cron-розкладу:

- H/5 * * * *: завдання виконується кожні 5 хвилин;
- 0 12 * * * *: завдання виконується щодня о 12:00;
- 30 2 * * 1-5: завдання виконується о 2:30 ночі кожен будній день (з понеділка по п'ятницю).

Особливості cron у Jenkins включають використання H (hashed), що дозволяє рівномірно розподіляти запуск завдань у зазначений інтервал.

Наприклад, ви можете налаштувати запуск кожні 10 хвилин за допомогою виразу `H/10 * * * *`, що забезпечить унікальний початок для кожного job. Також cron-стиль дає змогу здійснювати перевірки навіть у середовищах із закритими мережами, де Jenkins може перевіряти репозиторій за розкладом. Крім того, цей метод має високу гнучкість у налаштуванні періодичності.

До недоліків cron-стилю можна віднести затримки між подією та запуском завдання, які можуть тривати до кількох хвилин. Також наявність періодичних перевірок, навіть коли в репозиторії немає змін, може призводити до зайвих витрат ресурсів і збільшення навантаження на сервер.

Приклад у Jenkins Pipeline:

```
groovy

pipeline {
    agent any
    triggers {
        pollSCM('H/5 * * * *') // Перевірка змін у репозиторії кожні 5 хвилин
    }
    stages {
        stage('Build') {
            steps {
                echo 'Running periodic build...'
            }
        }
    }
}
```

Порівняння webhooks та cron-стилю:

Метод	Швидкість реакції	Ресурси сервера Jenkins	Гнучкість	Рекомендоване використання
Webhooks	Миттєва	Мінімальні	Висока	Для проєктів із частими змінами
Cron-стиль	Затримка (хвилини)	Помірні	Висока	Для періодичних завдань або обмежень мережі

Обидва методи можна комбінувати для створення гнучкої CI-системи. Наприклад, використовувати webhooks для швидкого запуску pipeline після push, а cron-стиль для періодичного виконання завдань, як-от оновлення залежностей чи перевірки безпеки.

Доступність Jenkins

Безперервна інтеграція залежить від належно налаштованого та стабільного мережевого середовища, оскільки різні компоненти CI-системи взаємодіють через мережу. У цьому контексті важливо врахувати такі аспекти, як доступність серверів, забезпечення безпеки з'єднань, управління мережевими фаєрволами та налаштування взаємодії між різними службами.

Для коректної роботи Jenkins, особливо в інтеграції з GitHub або іншими зовнішніми сервісами, сервер Jenkins має бути доступним для зовнішніх запитів. Це особливо важливо для використання webhooks, які надсилають запити на Jenkins для автоматичного запуску процесів CI.

Основні вимоги до доступності:

- *публічна IP-адреса або DNS*: сервер Jenkins повинен мати публічну IP-адресу або доменне ім'я (наприклад, jenkins.example.com), щоб приймати запити від зовнішніх сервісів, таких як GitHub. У випадках, коли Jenkins працює в локальній мережі, можна використовувати NAT (Network Address Translation) або проксі-сервер для забезпечення доступу;
- *використання тунелювання*: якщо Jenkins розміщений у локальній мережі без прямого доступу, можна використовувати інструменти тунелювання, такі як ngrok, щоб забезпечити тимчасовий доступ через публічний URL;
- *обмеження доступу*: щоб уникнути небажаного доступу, слід обмежити коло IP-адрес, які можуть звертатися до Jenkins. Наприклад, у фаєрволі можна дозволити лише запити з IP-адрес сервісів GitHub.

Приклад:

Якщо Jenkins розміщений за NAT, потрібно:

- прокинути порт (наприклад, 8080) із зовнішньої мережі до внутрішнього сервера Jenkins;
- використовувати DNS-ім'я для зручного доступу (jenkins.company.com → публічний IP).

Безпека

Безпека є критично важливим аспектом при інтеграції Jenkins у мережеве середовище, особливо якщо сервер Jenkins доступний із зовнішньої мережі.

Рекомендації щодо дотримання безпеки:

- *HTTPS-з'єднання*: усі з'єднання з Jenkins мають бути зашифрованими. Для цього використовується SSL-сертифікат. Можна отримати безкоштовний SSL-сертифікат від Let's Encrypt або придбати комерційний сертифікат;
- *аутентифікація та авторизація*: налаштуйте багаторівневу аутентифікацію для доступу до Jenkins. Наприклад, використовуйте плагіни Jenkins для інтеграції з LDAP або OAuth (GitHub, Google). Обмежте доступ до ресурсів Jenkins за ролями. Наприклад, адміністратори повинні мати доступ до налаштувань, а розробники – лише до своїх проектів.
- *захищені токени для Webhooks*: у налаштуваннях GitHub webhook додайте секретний токен (secret), який Jenkins перевірятиме під час кожного запиту. Увімкніть перевірку цього токена в Jenkins.
- *оновлення та патчі*: регулярно оновлюйте Jenkins та плагіни для усунення вразливостей. Використовуйте LTS-версії Jenkins.

Приклад налаштування HTTPS для Jenkins:

- встановити сертифікат на сервер Jenkins (через Nginx або Apache як проксі-сервер);
- увімкнути переадресацію з HTTP на HTTPS.

```

nginx

server {
    listen 80;
    server_name jenkins.example.com;
    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl;
    server_name jenkins.example.com;

    ssl_certificate /path/to/cert.pem;
    ssl_certificate_key /path/to/key.pem;

    location / {
        proxy_pass http://localhost:8080;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}

```

Проблеми з мережевими фаєрволами

У мережевому середовищі часто виникають проблеми через обмеження фаєрволів або інших систем безпеки, які блокують з'єднання між компонентами CI/CD.

Основні виклики:

- *закриті порти*: Jenkins за замовчуванням використовує порт 8080, але цей порт може бути закритим фаєрволом. Для роботи webhooks із GitHub потрібно відкрити порт 80 або 443 (для HTTPS);
- *з'єднання між Jenkins і сторонніми сервісами*: Jenkins може взаємодіяти з іншими сервісами, такими як SonarQube, Docker Registry, чи Kubernetes. Для цього потрібно переконатися, що всі необхідні порти відкриті;

- *налаштування NAT*: якщо Jenkins працює у внутрішній мережі, а SonarQube або інші сервіси розташовані зовні, потрібно налаштувати NAT або VPN для доступу;
- *доступ до інтернету*: Jenkins потребує доступу до інтернету для завантаження залежностей, оновлення плагінів і взаємодії з хмарними сервісами. У деяких компаніях доступ обмежується через проксі.

Рекомендується використовувати інструменти моніторингу, такі як Wireshark чи tcpdump, для діагностики проблем мережевого з'єднання. У налаштуваннях фаєрвола варто дозволити лише необхідні порти, зокрема HTTP/HTTPS (80/443), порт для Jenkins (8080, якщо він використовується) та порт для SonarQube (9000).

Приклад типового CI-середовища:

- *Jenkins*: доступний через HTTPS (443). Взаємодіє з GitHub через webhooks;
- *SonarQube*: працює на порті 9000. Отримує результати сканування від Jenkins через Sonar Scanner.
- *GitHub*: надсилає запити на сервер Jenkins через webhooks;
- *мережевий фаєрвол*: відкриті лише потрібні порти, а всі інші блокуються для забезпечення безпеки.

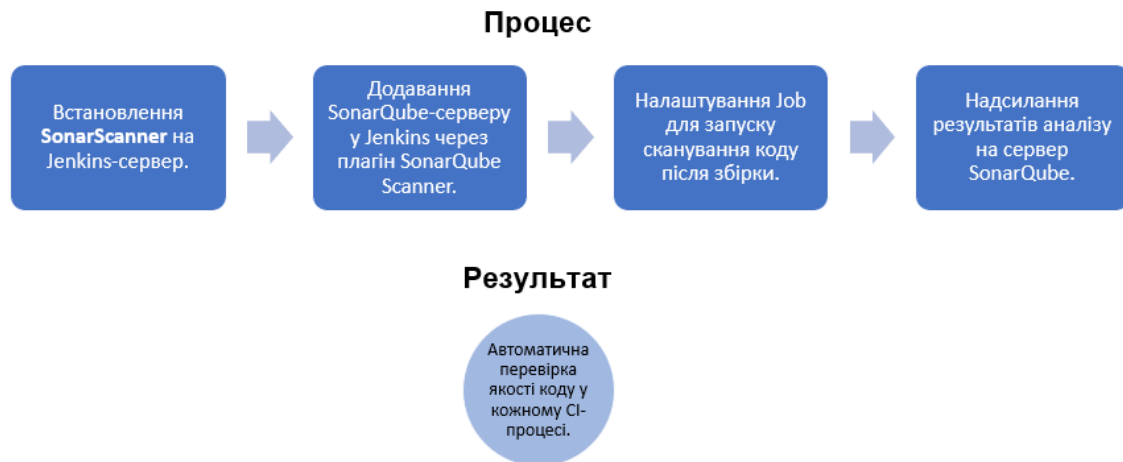
Коректна робота CI/CD системи залежить від налаштування мережевої доступності та безпеки. Забезпечення доступності Jenkins, шифрування з'єднань через HTTPS, а також оптимальна конфігурація фаєрволів і NAT допоможуть уникнути більшості потенційних проблем і підвищити надійність інтеграції.

Інтеграція SonarQube в інфраструктуру за допомогою Sonar-Scanner

SonarQube – це платформа для аналізу якості та безпеки коду, яка дозволяє знаходити баги, уразливості, кодову заборгованість і забезпечувати відповідність стандартам програмування. Його інтеграція в CI/CD-процес через

Sonar-Scanner допомагає підтримувати високу якість коду, автоматизуючи перевірки.

Інтеграція SonarQube через SonarScanner



Інтеграція SonarQube у CI/CD-процес складається з кількох основних кроків:

1) Інсталяція SonarQube:

- *локально*: установіть SonarQube на виділеному сервері або віртуальній машині. За замовчуванням SonarQube працює на порті 9000;
- *у хмарі*: використовуйте хмарні сервіси, наприклад, SonarCloud – SaaS-версію SonarQube.
- забезпечте доступність SonarQube для Jenkins та розробників через URL (наприклад, <http://sonarqube.example.com>).

2) Налаштування Jenkins для SonarQube: потрібно встановити плагін SonarQube Scanner for Jenkins через Jenkins Plugin Manager та додати сервер SonarQube у налаштуваннях Jenkins:

- увійдіть до Jenkins → Manage Jenkins → Configure System;
- у секції SonarQube servers додайте новий сервер, вказавши URL SonarQube та токен для аутентифікації.

3) Налаштування інструменту командного рядка для запуску аналізу коду Sonar-Scanner: для Jenkins рекомендується додати Sonar-Scanner як глобальний інструмент:

- увійдіть до Jenkins → Manage Jenkins → Global Tool Configuration;
- у секції SonarQube Scanner додайте новий інструмент, вказавши шлях до встановленого Sonar-Scanner.

4) Інтеграція з Jenkins Pipeline: потрібно розробити pipeline для автоматизації аналізу коду в процесі CI/CD.

Налаштування SonarQube та Sonar-Scanner

Для інсталяції SonarQube на сервер потрібно завантажити останню версію SonarQube з офіційного сайту, розпакувати архів та запустити сервер:

```
bash

./bin/linux-x86-64/sonar.sh start
```

Після чого слід відкрити SonarQube у браузері (<http://localhost:9000>) і створити токен доступу для Jenkins.

Для встановлення Sonar-Scanner потрібно завантажити Sonar-Scanner:

wget <https://binaries.sonarsource.com/Distribution/sonar-scanner-cli/sonar-scanner-cli-<version>.zip>

Розпакувати архів і додати шлях до sonar-scanner у змінну середовища PATH.

Приклад конфігурації sonar-project.properties для аналізу проєкту:

```
properties

sonar.projectKey=my_project
sonar.projectName=My Project
sonar.projectVersion=1.0
sonar.sources=src
sonar.language=java
sonar.sourceEncoding=UTF-8
sonar.host.url=http://sonarqube.example.com
sonar.login=<TOKEN>
```

Інтеграція SonarQube із Jenkins через Sonar-Scanner є ключовим елементом у забезпеченні якості коду. Використання Quality Gates та інтеграція з системами контролю версій дозволяють створити прозорий процес перевірки якості, знижуючи ризики впровадження небезпечного чи неякісного коду.

Автоматичні нотифікації

Автоматичні нотифікації є важливою частиною CI/CD-процесу, оскільки вони забезпечують швидке інформування команди про результати виконання завдань, збої чи інші події. Jenkins пропонує широкий спектр можливостей для інтеграції нотифікацій у різні канали.

Автоматичні нотифікації

Способи налаштування сповіщень:

Email:

- Відправка листа про успішне чи невдале збірку.

Slack:

- Інтеграція Jenkins із Slack для надсилання повідомлень у канали.

Telegram/Discord:

- Інтеграція через боти.

Переваги:

Оперативне інформування розробників про статус змін.



Email-нотифікації

Jenkins підтримує надсилання звітів і сповіщень через електронну пошту.

Для цього потрібно налаштувати SMTP-сервер:

- увійдіть до Jenkins → Manage Jenkins → Configure System;
- у секції Email Notification вкажіть SMTP-сервер (наприклад, smtp.gmail.com), порт (587), ім'я користувача та пароль.

Slack/Discord-нотифікації

Для інтеграції Slack або Discord використовується плагін Slack Notification Plugin.

Налаштування Slack:

- у Slack створіть новий Incoming Webhook для вашого робочого простору та отримайте URL для інтеграції;
- у Jenkins → Manage Jenkins → Configure System додайте налаштування Slack.

GitHub-нотифікації

Jenkins може автоматично відправляти статус виконання завдань (наприклад, Success, Failure) до pull request або commit в GitHub.

Для цього використовується плагін GitHub Integration Plugin.

Приклад конфігурації GitHub-нотифікації:


```
pipeline {
  agent any
  stages {
    stage('Build') {
      steps {
        echo 'Building project...'
      }
    }
  }
  post {
    success {
      githubNotify context: 'Build Status',
                  status: 'SUCCESS',
                  description: 'Build completed successfully.'
    }
    failure {
      githubNotify context: 'Build Status',
                  status: 'FAILURE',
                  description: 'Build failed.'
    }
  }
}
```

Нотифікації через Telegram

Jenkins також може інтегруватися з Telegram для надсилання сповіщень у групи або особисті чати.

Використовується плагін Telegram Notification Plugin:

- створіть бота в Telegram через @BotFather та отримайте токен;
- у Jenkins налаштуйте токен бота та ідентифікатор чату.

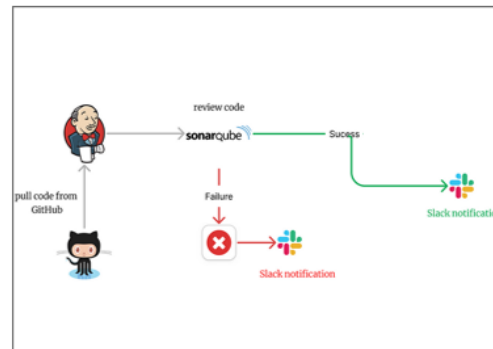
Використовуючи різні канали, такі як Email, Slack, Telegram, або GitHub, Jenkins забезпечує гнучкість і адаптивність нотифікацій відповідно до потреб проєкту. Інтеграція цих сповіщень у пайплайни сприяє підвищенню продуктивності команди та полегшує моніторинг процесів.

Послідовності роботи елементів у створеній CI

Безперервна інтеграція забезпечує автоматизацію ключових етапів розробки, включаючи тестування, перевірку коду та підготовку артефактів. Нижче наведено детальний опис етапів роботи елементів у розглянутій CI-структурі:

Послідовність роботи елементів CI

1. Розробник пушить зміни у GitHub репозиторій.
2. GitHub надсилає Webhook на Jenkins.
3. Jenkins отримує сигнал і запускає Job:
Збірка проекту.
Тестування (unit, integration tests).
Аналіз якості коду через SonarScanner.
4. Jenkins формує результати та надсилає нотифікації у Slack або поштою.
5. У разі успішного тестування — можливий деплой у тестове середовище.



Етапи роботи CI-процесу:

1) Розробник робить push у GitHub репозиторій:

- код із новими змінами надсилається до віддаленого репозиторію (branch або mainline);
- це є тригером для початку CI-процесу.

2) Webhook у GitHub відправляє запит на Jenkins:

- після події push, pull request чи merge, Webhook у GitHub надсилає HTTP-запит до Jenkins за заздалегідь налаштованим URL (наприклад, `http://<Jenkins_URL>/github-webhook/`);
- Webhook передає інформацію про зміни (комміти, автор, гілка тощо), що дозволяє Jenkins ініціювати відповідні завдання.

3) Jenkins запускає збірку (build) проекту:

- Jenkins ініціює визначений у Pipeline процес збірки. Процес може включати:
 - завантаження вихідного коду з репозиторію;
 - компіляцію проекту (для мов, таких як Java, C++ тощо);
 - виконання статичних і динамічних перевірок.
- конфігурація Pipeline може бути як Declarative, так і Scripted.

4) під час збірки виконується:

- статичний аналіз коду (SonarQube):

- SonarQube сканує код на наявність помилок, вразливостей, повторень і недоліків у стилі;
- Sonar-Scanner інтегрується через Jenkins Pipeline для автоматичного запуску після збірки;
- успішне проходження Quality Gate є критичним для подальших етапів;
- автоматичне тестування:
 - виконуються модульні, інтеграційні чи системні тести;
 - інструменти для тестування можуть включати JUnit, TestNG, Selenium, або PyTest залежно від технологій проекту;
 - звіти про результати тестів автоматично генеруються та додаються до Jenkins Dashboard.

5) Jenkins повідомляє про статус збірки через нотифікації:

- після завершення збірки Jenkins надсилає сповіщення:
 - у Slack або Telegram для команди розробників;
 - у GitHub у вигляді статусу завдання (success/failure) у pull request;
 - через Email для менеджерів або CI-адміністраторів.
- у разі помилки розробники отримують детальну інформацію про проблему (лог збірки, звіт тестів тощо).

6) У разі успіху створюється артефакт для подальшого використання (наприклад, у CD):

- артефактом може бути:
 - зібраний .jar, .war, .zip, або інший виконуваний файл;
 - докер-образ для контейнеризації програми;
 - статичний вебсайт або інші ресурси для деплоюменту;
- артефакт автоматично завантажується у репозиторій артефактів, наприклад, Nexus, Artifactory або Docker Hub та стає основою для наступного етапу в CI/CD – Continuous Deployment.

Інтеграція тестів і аналізу коду дозволяє швидко виявляти помилки на ранніх етапах розробки, що сприяє підвищенню якості продукту.

Автоматизація процесів мінімізує ручну роботу, знижує ризик людських помилок і значно пришвидшує виконання задач. Централізована звітність забезпечується через Jenkins, який надає єдине місце для моніторингу стану збірок, тестів і аналізу коду.

Готовність до деплоюменту досягається завдяки створенню збережених артефактів, які можна одразу використовувати в CD-процесі.

Послідовність роботи компонентів CI забезпечує автоматизований і структурований процес розробки, тестування та підготовки програмного забезпечення. Чітко визначені етапи та їх інтеграція через Jenkins, GitHub, SonarQube й інші інструменти створюють стабільну та надійну екосистему, яка сприяє підвищенню якості продукту та продуктивності команди.

Використання GitHub Action для побудови CI-системи

Розглянемо використання інструментів GitHub та GitHub Action для побудови мінімально укомплектованої CI-системи. З огляду на те, що наразі найкращим інструментом аналізу коду є SonarQube, як і в попередньому прикладі використаємо саме цей інструмент.

GitHub та GitHub Actions забезпечують зручну екосистему для автоматизації процесів CI/CD включаючи збірку, тестування та розгортання додатків, безпосередньо в репозиторії.

Для побудови CI-системи за допомогою GitHub Webhooks, яка інтегрується із SonarQube для аналізу якості коду, можна дотримуватись таких кроків:

1. Налаштування Webhooks в GitHub

Webhooks дозволяють GitHub автоматично повідомляти SonarQube або CI-систему про події в репозиторії (наприклад, push чи pull request). Для налаштування у репозиторії GitHub потрібно перейти до пункту Settings, обрати Webhooks та додати новий вебхук.

Наприклад, в даному випадку можуть бути наступні параметри:

- *Payload URL*: наприклад, URL сервера SonarQube `http://sonarqube.example.com/webhook`
- *Content type*: `application/json`.
- *Secret*: потрібно згенерувати секретний токен для перевірки автентичності;
- *Events*: слід обрати відповідні події `push` та/або `pull_request` залежно від потреб.

Потрібно перевірити, чи Webhook надсилає повідомлення. GitHub надає інструменти для діагностики у розділі Webhooks.

2. Налаштування GitHub Actions

Для роботи з GitHub Actions потрібно створити файл `.github/workflows/sonar.yml` у потрібному репозиторії для виконання автоматизованого аналізу коду з використанням SonarQube.

Нижче наведено фрагмент файлу конфігурації `sonar.yml`:

```

jobs:
  sonar_analysis:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout repository
        uses: actions/checkout@v3

      - name: Set up Java
        uses: actions/setup-java@v3
        with:
          java-version: 17

      - name: Cache SonarQube Scanner
        uses: actions/cache@v3
        with:
          path: ~/.sonar
          key: ${ runner.os }-sonar
          restore-keys: |
            ${ runner.os }-sonar

      - name: Run SonarQube Scanner
        env:
          SONAR_TOKEN: ${ secrets.SONAR_TOKEN }
        run: |
          sonar-scanner \
            -Dsonar.projectKey=<PROJECT_KEY> \
            -Dsonar.host.url=<SONARQUBE_SERVER_URL> \
            -Dsonar.login=${ secrets.SONAR_TOKEN }

```

3. Налаштування SonarQube

Для інтеграції інструменту аналізу коду в систему потрібно створити проєкт у SonarQube та згенерувати токен аутентифікації для GitHub Actions.

Відповідний токен потрібно додати до GitHub Secrets: у репозиторії GitHub потрібно перейти до налаштування (Settings) та в пункті Secrets and variables обрати Actions і додати секрет SONAR_TOKEN із токеном доступу SonarQube.

4. Інтеграція Webhooks та GitHub Actions

GitHub Webhooks працюють як тригери для запуску GitHub Actions при подіях (push, pull_request). Аналіз коду виконується автоматично через конфігурацію sonar.yml.

5. Перевірка результатів у SonarQube

Після виконання GitHub Actions результати аналізу коду з'являться у проекті SonarQube.

6. Додавання сповіщень у CI-систему

Сповіщення дозволять оперативно інформувати команду про статус аналізу коду, результати Quality Gate та інші події. Як і попередньому прикладі, для сповіщень можна використати різні сервіси: Microsoft Teams, Telegram, Slack, Email тощо.

Так, наприклад, для інтеграції зі Slack потрібно:

- увійти у Slack і у розділі Apps обрати пункт Incoming Webhooks;
- створити новий Webhook та обрати канал, куди надсилатимуться сповіщення;
- скопіювати URL Webhook та додати до GitHub Secrets: для цього у GitHub репозиторії потрібно перейти до налаштувань (Settings) обрати пункт Secrets and variables та Actions відповідної додати секрет.

Далі потрібно оновити конфігурацію у файлі .github/workflows/sonar.yml, так , щоб включити сповіщення через Slack.

Для використання інших сервісів для сповіщень потрібно замінити Webhook відповідним API або інтеграцією та налаштувати конфігурацію у відповідному yaml-файлі.

Автоматичні сповіщення про результати аналізу в реальному часі дозволяють команді швидко реагувати на помилки або проблеми з якістю коду.

Послідовності роботи елементів у створеній CI

Розглянемо основні етапи роботи у запропонованій CI-структурі:

Процес роботи створеної CI системи являє собою комплексний автоматизований потік, який починається з дій розробника. Коли розробник вносить зміни до кодової бази через push команду або створює Pull Request у GitHub репозиторії, це ініціює цілий ланцюг автоматизованих процесів. GitHub, отримавши такі зміни, миттєво активує налаштований webhook, який служить тригером для подальших дій.

Активований webhook передає керування до GitHub Actions, де починається виконання попередньо налаштованого workflow, визначеного у конфігураційному файлі sonar.yml. На цьому етапі GitHub Actions готує необхідне середовище для проведення аналізу коду, завантажуючи всі потрібні залежності та інструменти.

Наступним кроком у послідовності є взаємодія з системою аналізу якості коду SonarQube, яка має наступні кроки:

- GitHub Actions передає код до SonarQube для аналізу;
- SonarQube проводить статичний аналіз коду;
- Результати аналізу зберігаються в SonarQube;
- SonarQube повертає результати до GitHub Actions.

SonarQube проводить комплексну перевірку коду, включаючи пошук потенційних помилок, оцінку якості коду, перевірку дотримання стандартів кодування та багато інших метрик. Всі результати аналізу зберігаються в базі даних SonarQube для подальшого використання та порівняльного аналізу з попередніми версіями коду.

Після завершення аналізу GitHub Actions отримує детальний звіт від SonarQube і на його основі оновлює статус перевірки безпосередньо в Pull Request або коміті. Це дозволяє іншим членам команди миттєво бачити, чи пройшов код необхідні перевірки якості. На базі цих результатів система також генерує інформативне сповіщення, яке містить ключові метрики та виявлені проблеми.

Важливим елементом процесу є система сповіщень, яка забезпечує оперативне інформування команди. У даному випадку налаштована інтеграція зі Slack, через який команда отримує миттєві повідомлення про результати аналізу. Це дозволяє розробникам швидко реагувати на виявлені проблеми та підтримувати високу якість коду.

У випадку, якщо в процесі аналізу були виявлені проблеми або код не відповідає встановленим стандартам якості, запускається цикл виправлень. Розробники отримують детальну інформацію про виявлені проблеми, вносять необхідні виправлення у код та роблять новий коміт. Цей новий коміт автоматично запускає весь процес аналізу знову, створюючи таким чином безперервний цикл покращення якості коду.